Plan de implementación: alta disponibilidad en el acceso a una base de datos

Clúster PostgreSQL de 3 nodos con Patroni, etcd y HAProxy sobre Docker macOS (Apple Silicon) + 2 × Pop!_OS · Octubre 2026

Índice

1. Requerimientos y cómo los cumple este diseño
2. Arquitectura
3. Decisiones de diseño (y por qué)
4. Inventario y convenciones
5. Fase 0: Preparación del sistema operativo
6. Fase 1: Red entre las 3 máquinas
7. Fase 2: Instalación de Docker
8. Fase 3: Repositorio y archivos del proyecto
9. Fase 4: Clúster etcd
10. Fase 5: PostgreSQL + Patroni
11. Fase 6: HAProxy
12. Fase 7: Base de datos, esquema y usuario de la aplicación
13. Fase 8: Generador de carga, monitor y verificador
14. Fase 9: Pruebas de falla
15. Fase 10: Guion de la demostración en vivo
16. Fase 11: Informe y evidencias
17. Solución de problemas
18. Comandos de referencia rápida
19. Preguntas probables del catedrático
20. Cronograma sugerido y checklist final
21. Laboratorio local: los 3 nodos en una sola máquina

> **Atajo:** el repositorio `github.com/NohelEstradaP/ha_db_ip` ya trae todos los archivos de este plan, un instalador (`instalar.sh`) que hace las fases 0 a 3 en cada equipo y scripts para las fases 4 a 7. Ver §8.1. Los pasos manuales del plan sirven como referencia y para explicar qué hace cada script.

1. Requerimientos y cómo los cumple este diseño

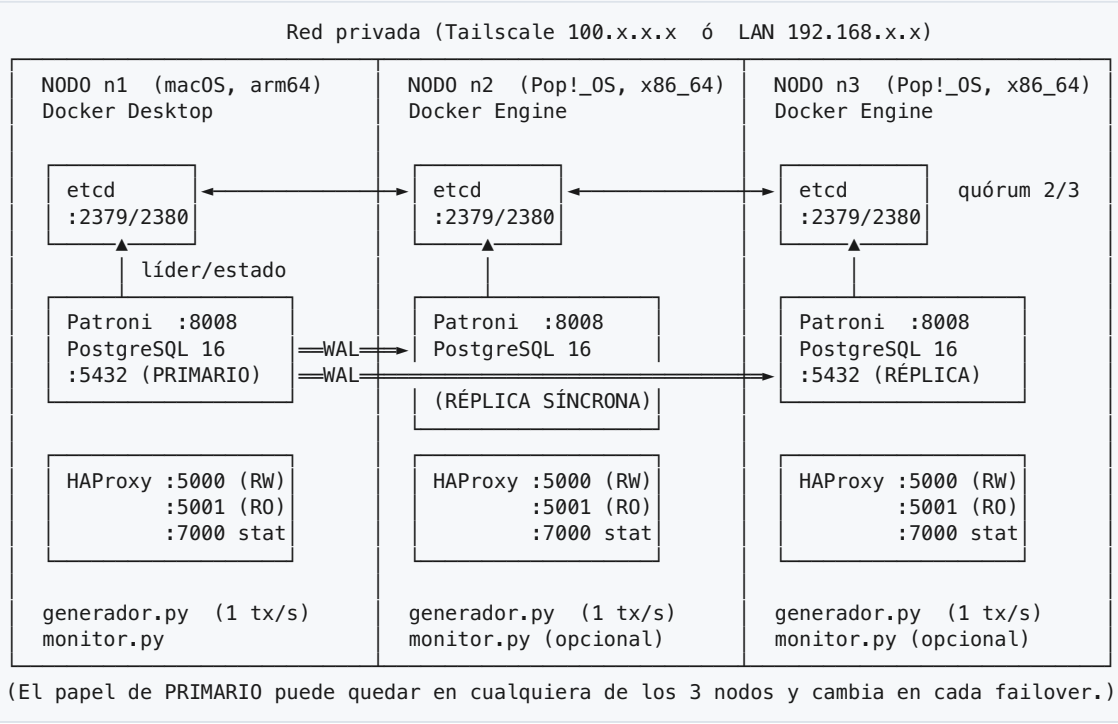
#	Requerimiento	Cómo se cumple
R1	La BD recibe **al menos 90 cambios por minuto**	Un generador en cada nodo hace 1 transacción/segundo. Cada transacción tiene 3 cambios (1 `INSERT` + 2 `UPDATE`). Con los 3 nodos: 180 transacciones y 540 cambios por minuto. Con un nodo caído siguen 120 transacciones y 360 cambios por minuto, siempre más de 90.
R2	Acceso desde **dos o más equipos**, siempre con algún **mirror**	3 nodos PostgreSQL: 1 primario y 2 réplicas en *streaming replication*. Una réplica es **síncrona**, así que cada `COMMIT` confirmado ya existe en al menos 2 máquinas. Se puede leer y escribir desde cualquiera de los 3 equipos.
R3	En algún momento **uno de los equipos se apagará** (cualquiera, sin aviso)	Patroni y etcd detectan la caída. Si cae una réplica, no pasa nada visible. Si cae el primario, se elige una réplica nueva como primario en unos 20 a 30 s, sin intervención manual.
R4	La **integridad y la funcionalidad continúan** con un equipo desconectado	Con replicación síncrona no se pierde ninguna transacción confirmada (RPO = 0). El quórum de etcd (2 de 3) evita el *split-brain*. Los reintentos de la aplicación son idempotentes, así que no hay duplicados. Un verificador comprueba invariantes: la suma de saldos se mantiene y no faltan secuencias.
R5	Al **restablecer la comunicación se vuelve a sincronizar**	El nodo que regresa arranca como réplica. Si era el primario viejo, Patroni le aplica `pg_rewind` y lo pone a seguir al primario nuevo. Si no, recupera el WAL atrasado. Todo es automático.

Restricción del entorno: 3 máquinas Unix (1 Mac con Apple Silicon y 2 con Pop!_OS), que pueden estar en la misma red o en redes distintas.

---

2. Arquitectura

2.1 Vista general



2.2 Componentes y responsabilidad

Componente	Versión	Qué hace	Puertos
etcd	3.5.17	Almacén de consenso distribuido (Raft). Guarda quién es el líder del clúster de BD. Necesita mayoría (2 de 3) para aceptar cambios.	2379 (clientes), 2380 (peers)
PostgreSQL	16	El motor de base de datos.	5432
Patroni	4.0.4	Agente que corre junto a cada PostgreSQL. Lo arranca y lo detiene, configura la replicación, hace el failover automático, aplica `pg_rewind` y expone una API REST de salud.	8008
HAProxy	3.0	Balanceador TCP. Consulta la API de Patroni y manda el puerto 5000 siempre al primario y el 5001 a las réplicas. Sirve para clientes que no soportan varios hosts (pgAdmin, DBeaver, etc.).	5000, 5001, 7000
Tailscale (recomendado)	actual	VPN *mesh* que le da a cada máquina una IP fija `100.x.y.z` que no cambia aunque cambie la red física.	UDP 41641
Aplicación Python	3.10+	`generador.py` (carga), `monitor.py` (estado en vivo) y `verificar.py` (integridad). Usa `psycopg` 3.	n/a

2.3 Cómo llega la aplicación al primario

Hay dos caminos y los dos se mantienen activos:

1. **Cadena de conexión con varios hosts (libpq)**. Es el camino principal de la aplicación: `host=IP_n1,IP_n2,IP_n3 port=5432 target_session_attrs=read-write` libpq prueba los hosts en orden y se queda con el que acepta escrituras. No depende de ningún componente intermedio y funciona aunque cualquiera de los 3 nodos esté apagado.

2. **HAProxy local** (`localhost:5000` en cualquier nodo). Sirve para herramientas gráficas y para la demostración. Cada HAProxy pregunta a los 3 Patroni (`GET /primary`) y solo enruta al que responde 200.

> No se usa una IP virtual (VIP/keepalived) porque no funciona entre redes distintas ni con Docker Desktop en macOS. Las dos opciones anteriores la sustituyen.

2.4 Flujo de un failover (cae el primario)

```
t=0s    n1 (primario) se apaga de golpe.
t=0–20s La llave de líder de n1 en etcd sigue vigente (ttl=20s). Los COMMIT pendientes fallan
        y los generadores de n2 y n3 entran en reintento.
t≈20s   La llave expira. Patroni en n2 y n3 compara el WAL recibido. Gana la réplica más
        actualizada (la síncrona), que toma la llave de líder en etcd (con quórum de n2+n3).
t≈21s   n2 ejecuta pg_promote: pasa a PRIMARIO en un timeline nuevo.
        n3 se reconfigura para replicar desde n2. Patroni elige n3 como nueva réplica síncrona.
t≈22s   libpq/HAProxy encuentran el nuevo primario. Los generadores reintentan la MISMA secuencia
        (es idempotente) y recuperan el atraso.
...     n1 vuelve. Patroni detecta que su timeline quedó atrás, ejecuta pg_rewind para descartar
        lo que nunca se confirmó y arranca n1 como RÉPLICA de n2. Se pone al día en segundos.
```

---

3. Decisiones de diseño (y por qué)

1. **Tres nodos, no dos.** Con 2 nodos, si se pierde la comunicación cada uno cree que el otro murió y ambos se declaran primario: es *split-brain* y los datos divergen. Con 3 hay mayoría (2 de 3) y solo el lado con mayoría puede tener primario. El nodo aislado sabe que está en minoría y se degrada a solo lectura.
2. **Replicación síncrona (`synchronous_mode: true`).** Un `COMMIT` solo se confirma al cliente cuando la réplica síncrona ya recibió el WAL y lo escribió a disco. Si el primario muere, la réplica síncrona tiene todo lo confirmado, así que RPO = 0. `synchronous_mode_strict: false` permite que, si quedara un solo nodo vivo, siga aceptando escrituras en lugar de congelarse. Eso solo pasaría con 2 caídas, que está fuera del alcance del requerimiento.
3. **Docker en las 3 máquinas.** Da la misma imagen Debian, el mismo PostgreSQL y la misma glibc en macOS y en Linux. Esto importa: la replicación física copia los archivos de datos byte a byte, y si un nodo usara las colaciones de macOS y otro las de glibc los índices de texto podrían corromperse. Además el `docker-compose.yml` es idéntico en los 3 nodos y solo cambia el archivo `.env`.
4. **Locale `C.UTF-8` al inicializar.** El orden de texto no depende de la versión de glibc ni de la arquitectura, lo que elimina el riesgo de colación entre arm64 y x86_64.
5. **`pg_rewind` y `data-checksums`.** Permiten que el primario viejo vuelva como réplica sin copiar toda la BD de nuevo. Los *checksums* detectan corrupción de páginas.
6. **Tailscale como red por defecto.** Cada máquina recibe una IP fija `100.x.y.z` que funciona igual en la misma LAN (conexión directa) y en redes distintas. etcd y Patroni guardan direcciones IP, y cambiarlas después obliga a reconstruir etcd (§6.4). Con Tailscale ese problema no existe.
7. **Aplicación idempotente.** Cada transacción lleva la llave única `(generador, seq)`. Si la conexión se cae durante el `COMMIT` no se sabe si se guardó. Se reintenta la misma `seq` y `ON CONFLICT DO NOTHING` impide duplicarla. Así se tienen "exactamente una vez" y ningún hueco.
8. **Tiempos de Patroni ajustados** (`ttl: 20`, `loop_wait: 5`, `retry_timeout: 5`). El failover baja de unos 40 s (valores por defecto) a 20–30 s. Se cumple la regla `loop_wait + 2 × retry_timeout ≤ ttl` (5 + 10 = 15 ≤ 20).
9. **Tiempos de etcd para WAN** (`heartbeat 250 ms`, `election 2500 ms`). Toleran la latencia de Tailscale entre redes distintas sin elecciones falsas.

---

4. Inventario y convenciones

4.1 Tabla de nodos (llenar antes de empezar)

Nodo	Máquina	SO	Arquitectura	Hostname	IP Tailscale	IP LAN (si aplica)
n1	Mac	macOS (Apple Silicon)	arm64	`_____`	`100.___.___.___`	`192.168.___.___`
n2	PC 1	Pop!_OS	x86_64	`_____`	`100.___.___.___`	`192.168.___.___`
n3	PC 2	Pop!_OS	x86_64	`_____`	`100.___.___.___`	`192.168.___.___`

> En este documento `N1_IP`, `N2_IP` y `N3_IP` representan estas IP. Se usa **una sola** familia de IP para todo (Tailscale o LAN), nunca mezcladas.

4.2 Requisitos mínimos por máquina

- 4 GB de RAM libres para Docker (2 GB alcanzan para la carga de la práctica), 10 GB de disco libre y 2 núcleos.
- Usuario con `sudo`.
- Puertos libres: **2379, 2380, 5432, 5000, 5001, 7000, 8008**. Si hay un PostgreSQL instalado localmente usando el 5432, hay que detenerlo (ver §5.4).

4.3 Contraseñas

Defínanlas una vez y úsenlas iguales en los 3 `.env`. Nunca se suben a GitHub.

Variable	Uso
`PG_SUPERUSER_PASSWORD`	usuario `postgres`
`PG_REPLICATION_PASSWORD`	usuario `replicator` (streaming)
`PG_REWIND_PASSWORD`	usuario `rewind_user` (pg_rewind)
`APP_PASSWORD`	usuario `app` (generador, monitor y verificador)

Para generarlas: `openssl rand -base64 18 | tr -d '/+='`

---

5. Fase 0: Preparación del sistema operativo

5.1 macOS (n1)

1. **Actualizar** el sistema y tener las *Command Line Tools*: `bash xcode-select --install # si ya están instaladas, avisa y no hace nada`
2. **Homebrew** (si no está): `bash /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"`
3. **Herramientas**: `bash brew install git python@3.12 netcat jq`
4. **Desactivar el reposo.** Esto es crítico: si la Mac se duerme, para el clúster es lo mismo que apagarla. - *Ajustes del Sistema* → *Batería* → *Opciones* → activar "Evitar el reposo automático en el adaptador de corriente cuando la pantalla esté apagada". - Terminal: `bash sudo pmset -c sleep 0 disksleep 0 displaysleep 30 pmset -g | grep -E ' sleep|disksleep'` - Durante pruebas y demo, dejar corriendo `caffeinate -dimsu` en una terminal aparte. - **No cerrar la tapa** salvo que esté conectada a un monitor externo con cargador.
5. **Hora sincronizada:** *Ajustes* → *General* → *Fecha y hora* → "Ajustar automáticamente".
6. **Firewall de macOS:** si está activo, permitir conexiones entrantes a *Docker* y *Tailscale* cuando lo pregunte (*Ajustes* → *Red* → *Firewall* → *Opciones*).

5.2 Pop!_OS (n2 y n3)

1. **Actualizar**: `bash sudo apt update && sudo apt full-upgrade -y sudo apt install -y git curl ca-certificates gnupg jq netcat-openbsd python3 python3-venv python3-pip`
2. **Hostname** (opcional pero ayuda a la claridad): `bash sudo hostnamectl set-hostname popos-n2 # en la otra: popos-n3`
3. **Desactivar suspensión:** - *Settings* → *Power* → "Automatic Suspend" en *Off*. - Terminal (evita la suspensión por completo): `bash sudo systemctl mask sleep.target suspend.target hibernate.target hybrid-sleep.target` - Si es laptop, cerrar la tapa no debe suspender: `bash sudo sed -i 's/^#\?HandleLidSwitch=.*/HandleLidSwitch=ignore/' /etc/systemd/logind.conf sudo sed -i 's/^#\?HandleLidSwitchExternalPower=.*/HandleLidSwitchExternalPower=ignore/' /etc/systemd/logind.conf sudo systemctl restart systemd-logind`
4. **Hora sincronizada:** `bash sudo timedatectl set-ntp true timedatectl status | grep -E 'synchronized|NTP'`
5. **Firewall (ufw).** En Pop!_OS viene desactivado. Verificar con: `bash sudo ufw status` Si está **activo**, abrir los puertos solo para la red del clúster: `bash # Modo Tailscale: sudo ufw allow in on tailscale0 # Modo LAN (ajustar la subred): for p in 2379 2380 5432 5000 5001 7000 8008; do sudo ufw allow from 192.168.1.0/24 to any port $p proto tcp; done`

   > Docker publica puertos con sus propias reglas de iptables y puede saltarse ufw. Aun así se recomienda la regla anterior para que las pruebas con `nc` sean coherentes.

5.3 Verificaciones comunes (las 3)

```
date                # las 3 deben marcar la misma hora (±1 s)
uname -m            # n1: arm64   n2/n3: x86_64
python3 --version   # 3.10 o superior
git --version
```

5.4 Verificar puertos libres (las 3)

```
# macOS
for p in 2379 2380 5432 5000 5001 7000 8008; do lsof -nP -iTCP:$p -sTCP:LISTEN && echo "OCUPADO $p";
done
# Pop!_OS
for p in 2379 2380 5432 5000 5001 7000 8008; do sudo ss -ltnp "sport = :$p" | grep -q LISTEN && echo
"OCUPADO $p"; done
```

Si el 5432 está ocupado por un PostgreSQL local: - macOS (Homebrew): `brew services stop postgresql@16` (o la versión instalada). - Pop!_OS: `sudo systemctl disable --now postgresql`.

Criterio de salida de la fase 0: las 3 máquinas sin reposo, con hora sincronizada y con los puertos libres.

---

6. Fase 1: Red entre las 3 máquinas

6.1 Modo recomendado: Tailscale (sirve para misma red y redes distintas)

1. Crear una cuenta gratuita en tailscale.com (una sola cuenta para las 3 máquinas, o compartir el *tailnet*).
2. **macOS (n1):** `bash brew install --cask tailscale` Abrir la app *Tailscale*, iniciar sesión y aceptar la extensión de red del sistema.
3. **Pop!_OS (n2, n3):** `bash curl -fsSL https://tailscale.com/install.sh | sh sudo tailscale up # Abrir la URL que imprime e iniciar sesión con la misma cuenta sudo systemctl enable --now tailscaled`
4. **Obtener las IP** en cada máquina y anotarlas en la tabla §4.1: `bash tailscale ip -4`
5. **Desactivar la expiración de llaves.** En la consola web de Tailscale, *Machines* → cada máquina → ⋯ → *Disable key expiry*. Si no, a los 180 días la máquina sale de la red sin aviso.
6. **Probar conectividad** desde cada nodo hacia los otros dos: `bash tailscale ping <IP_otro_nodo> # debe decir "via <ip>:<puerto>" (directo) o "via DERP" (relay) ping -c 3 <IP_otro_nodo> tailscale netcheck # muestra si UDP sale; útil en redes universitarias` - "via DERP" significa que la red bloquea UDP directo y el tráfico pasa por un relay. Funciona igual, con más latencia (50–200 ms). Para 180 transacciones/min es suficiente.

6.2 Modo alternativo: misma LAN sin VPN

Usarlo solo si no se puede instalar Tailscale.

1. Fijar la IP de cada máquina: - **Opción A (preferida):** reserva DHCP en el router (MAC → IP fija). - **Opción B:** IP estática.
    - Pop!_OS: *Settings → Network → (interfaz) → IPv4 → Manual*.
    - macOS: *Ajustes → Red → (interfaz) → Detalles → TCP/IP → Configurar IPv4: Manualmente*.
2. Verificar `ping` entre las 3.
3. Si es WiFi de universidad o de hotel, comprobar que no haya **aislamiento de clientes** (el ping entre equipos falla). En ese caso hay que usar Tailscale o un router propio.

6.3 Plan B de red el día de la demo

Hay que preparar las tres opciones y probarlas antes:

Situación el día de la demo	Acción
Hay Internet (cualquier red)	Tailscale. No se cambia nada.
Red sin Internet pero las máquinas se ven	Tailscale **no** sirve sin Internet al iniciar sesión. Si ya estaban conectadas y la red se cae, las conexiones directas existentes pueden seguir, pero no hay que confiar en eso. Usar el plan de abajo.
Sin red utilizable	Llevar un **router WiFi propio** o usar el *hotspot* de un celular. Las 3 máquinas se conectan ahí y Tailscale funciona sobre esa red (el hotspot tiene Internet).

> Recomendación fuerte: llevar un router de viaje o tener el hotspot listo, y que **las 3 máquinas ya hayan iniciado sesión en Tailscale** antes de llegar.

6.4 Procedimiento si cambian las IP (solo modo LAN)

etcd guarda las URL de sus *peers* con IP. Si las IP cambian, el clúster etcd no arranca y hay que reconstruirlo. Los **datos de PostgreSQL se conservan**.

1. Antes del cambio, anotar quién es el primario (`patronictl list`, ver §18).
2. En las 3: `docker compose down` (no usar `-v`).
3. En las 3: borrar **solo** el volumen de etcd: `docker volume rm hadb_etcd-data`.
4. Editar `.env` con las IP nuevas en las 3.
5. En las 3: `docker compose up -d etcd` y verificar la salud de etcd (§9.3).
6. **Primero** en el nodo que era primario: `docker compose up -d patroni`. Esperar a que aparezca como `Leader`.
7. Luego en los otros dos: `docker compose up -d patroni haproxy`.
8. Finalmente `docker compose up -d haproxy` en el primero.

Criterio de salida de la fase 1: cada nodo hace `ping` a los otros dos por la IP elegida y las 3 IP están anotadas.

---

7. Fase 2: Instalación de Docker

7.1 macOS (n1): Docker Desktop para Apple Silicon

1. Descargar *Docker Desktop for Mac (Apple Silicon)* desde docker.com e instalar. Alternativa: `brew install --cask docker`.
2. Abrirlo y en *Settings*: - *General* → "Start Docker Desktop when you sign in" **activado** (para que el nodo vuelva solo tras reiniciar). - *Resources* → Memory ≥ 4 GB, CPUs ≥ 2. - *General* → "Use Rosetta for x86_64/amd64 emulation" **activado** (solo se usa en el plan B de §10.6).
3. Verificar: `bash docker version docker compose version docker run --rm hello-world`

7.2 Pop!_OS (n2, n3): Docker Engine desde el repositorio oficial

Pop!_OS está basado en Ubuntu, así que se usa el repositorio de Ubuntu con el codename correcto:

```
# Quitar paquetes viejos que choquen (no pasa nada si no existen)
for pkg in docker.io docker-doc docker-compose podman-docker containerd runc; do sudo apt remove -y
$pkg; done

sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

. /etc/os-release
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://
download.docker.com/linux/ubuntu ${UBUNTU_CODENAME:-$VERSION_CODENAME} stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

sudo systemctl enable --now docker containerd     # arranque automático al encender
sudo usermod -aG docker "$USER"                   # usar docker sin sudo
```

Cerrar sesión y volver a entrar (o `newgrp docker`) y verificar:

```
docker version
docker compose version
docker run --rm hello-world
```

7.3 Prueba de red desde dentro de contenedores (las 3)

Docker Desktop en macOS y las reglas NAT de Linux pueden comportarse distinto con la VPN. Antes de seguir, en **cada** nodo se abre un puerto de prueba y se verifica desde los otros dos.

En el nodo que escucha (por ejemplo n1):

```
docker run --rm -p 5432:5432 alpine sh -c "while true; do echo hola | nc -l -p 5432; done"
```

Desde los otros dos nodos (en el host y dentro de un contenedor):

```
nc -vz <IP_n1> 5432
docker run --rm alpine nc -vz -w 3 <IP_n1> 5432
```

Probar también el **acceso a la propia IP** (*hairpin*) desde el mismo nodo, porque Patroni y HAProxy se consultan a sí mismos:

```
docker run --rm alpine nc -vz -w 3 <IP_propia> 5432
```

Repetir con n2 y n3 como servidor. Las 9 combinaciones deben decir `open`. Cortar el contenedor de prueba con `Ctrl+C`.

> Si la Mac **no** acepta conexiones entrantes por la IP de Tailscale hacia Docker Desktop, revisar el firewall de macOS (§5.1) y §17.

Criterio de salida de la fase 2: `hello-world` corre en las 3 máquinas y la prueba de puertos da `open` en las 9 combinaciones.

---

8. Fase 3: Repositorio y archivos del proyecto

8.1 Uso de GitHub para distribuir la configuración

El mismo repositorio se clona en las 3 máquinas. Así cualquier cambio se hace una vez y se sincroniza con `git pull`.

Instalación automática (recomendada)

El repositorio ya contiene todos los archivos de las secciones 8.3 a 8.14. En **cada** equipo:

```
git clone https://github.com/NohelEstradaP/ha_db_ip.git ~/hadb
cd ~/hadb
./instalar.sh           # modo Tailscale;  ./instalar.sh --lan  para solo red local
```

`instalar.sh` funciona en macOS y en Pop!_OS, y se puede volver a correr. Hace lo siguiente:

Paso	Qué hace	Sección manual equivalente
Paquetes	git, jq, Python, curl (Homebrew o apt)	§5.1, §5.2
Docker	Docker Desktop (Mac) o Docker Engine desde el repo oficial (Pop!_OS), arranque automático y grupo `docker`	§7
Tailscale	Instala, pide iniciar sesión y espera la IP `100.x`	§6.1
Energía y hora	Desactiva la suspensión y activa NTP	§5
Puertos	Avisa si 2379, 2380, 5432, 5000, 5001, 7000 u 8008 están ocupados	§5.4
`.env`	Pregunta nodo (`n1` / `n2` / `n3`) e IP de los 3. En el **primer** equipo genera las 4 contraseñas y las muestra; en los otros dos se pegan	§8.4
Firewall	Si `ufw` está activo, permite el tráfico desde las IP de los 3 nodos	§5.2
Python e imágenes	Crea `.venv` , instala `psycopg` , construye la imagen de Patroni y descarga etcd y HAProxy	§9.1, §13.1

Después, los scripts de `scripts/` cubren el resto:

Script	Uso	Sección
`./scripts/probar_puertos.sh`	Conectividad entre los 3 nodos	§7.3
`./scripts/iniciar.sh`	Levanta el nodo, espera quórum, primario y que el nodo esté sincronizado. Si el nodo fue reinstalado, lo reincorpora solo a etcd	§9, §10, §11
`./scripts/crear_bd.sh`	Usuario `app` , base `hadb` y esquema (una vez, en cualquiera)	§12
`./scripts/estado.sh`	Patroni, etcd y quién es primario	§18
`./scripts/panel.sh`	Panel web en tiempo real (`http://<IP>:8080`), con botón para elegir el primario; `--fondo` para segundo plano	§13.3
`./scripts/carga.sh iniciar`	Generador de este equipo en segundo plano (`detener` , `estado`)	§13.3
`./scripts/pctl.sh <args>`	`patronictl` (list, history, switchover, reinit…)	§18

Script	Uso	Sección
`source scripts/env_app.sh`	Prepara la terminal para `app/*.py`	§13.2

Repositorio propio (si se parte de cero)

```
# En la máquina donde se crean los archivos (por ejemplo la Mac):
mkdir -p ~/hadb && cd ~/hadb
git init -b main
# ...crear los archivos de §8.3 a §8.10...
git add . && git commit -m "Configuración inicial del clúster HA"
git remote add origin git@github.com:<usuario>/<repo>.git
git push -u origin main

# En n2 y n3:
git clone git@github.com:<usuario>/<repo>.git ~/hadb
```

- El archivo **`.env` NO se sube** (tiene contraseñas e IP). Se sube `.env.example` y cada nodo crea su `.env`.
- Revisar la identidad de git en cada máquina antes del primer commit: `git config --global user.name` y `git config --global user.email`.

8.2 Estructura

```
hadb/
├── .gitignore
├── .env.example
├── docker-compose.yml
├── patroni/
│   ├── Dockerfile
│   └── patroni.yml
├── haproxy/
│   └── haproxy.cfg
├── sql/
│   └── esquema.sql
├── app/
│   ├── requirements.txt
│   ├── generador.py
│   ├── monitor.py
│   └── verificar.py
└── scripts/
    └── probar_puertos.sh
```

8.3 `.gitignore`

```
.env
.venv/
__pycache__/
*.csv
evidencias/
docker-compose.override.yml
```

8.4 `.env.example` (copiar a `.env` en cada nodo y editar)

```
# ---- Identidad de ESTE nodo (cambia en cada máquina) ----
NODE_NAME=n1                # n1 | n2 | n3
NODE_IP=100.64.0.1          # IP de ESTE nodo (Tailscale o LAN)

# ---- IP de los 3 nodos (idénticas en las 3 máquinas) ----
N1_IP=100.64.0.1
N2_IP=100.64.0.2
N3_IP=100.64.0.3
```

```
# ---- Contraseñas (idénticas en las 3 máquinas) ----
PG_SUPERUSER_PASSWORD=CambiarEsto1
PG_REPLICATION_PASSWORD=CambiarEsto2
PG_REWIND_PASSWORD=CambiarEsto3
APP_PASSWORD=CambiarEsto4
```

En cada nodo:

```
cd ~/hadb
cp .env.example .env
nano .env          # NODE_NAME y NODE_IP propios; N1/N2/N3 y contraseñas iguales en los 3
```

8.5 `docker-compose.yml` (idéntico en los 3 nodos)

```
name: hadb

services:
  etcd:
    image: gcr.io/etcd-development/etcd:v3.5.17
    container_name: etcd
    restart: unless-stopped
    command:
      - /usr/local/bin/etcd
      - --name=${NODE_NAME}
      - --data-dir=/etcd-data
      - --listen-client-urls=http://0.0.0.0:2379
      - --advertise-client-urls=http://${NODE_IP}:2379
      - --listen-peer-urls=http://0.0.0.0:2380
      - --initial-advertise-peer-urls=http://${NODE_IP}:2380
      - --initial-cluster=n1=http://${N1_IP}:2380,n2=http://${N2_IP}:2380,n3=http://${N3_IP}:2380
      - --initial-cluster-state=new
      - --initial-cluster-token=hadb-etcd
      - --heartbeat-interval=250
      - --election-timeout=2500
    ports:
      - "2379:2379"
      - "2380:2380"
    volumes:
      - etcd-data:/etcd-data

  patroni:
    build: ./patroni
    image: hadb/patroni:16
    container_name: patroni
    restart: unless-stopped
    init: true
    depends_on:
      - etcd
    stop_grace_period: 60s
    shm_size: 256mb
    environment:
      PATRONI_NAME: ${NODE_NAME}
      PATRONI_SCOPE: hadb
      PATRONI_ETCD3_HOSTS: "${N1_IP}:2379,${N2_IP}:2379,${N3_IP}:2379"
      PATRONI_RESTAPI_LISTEN: 0.0.0.0:8008
      PATRONI_RESTAPI_CONNECT_ADDRESS: ${NODE_IP}:8008
      PATRONI_POSTGRESQL_LISTEN: 0.0.0.0:5432
      PATRONI_POSTGRESQL_CONNECT_ADDRESS: ${NODE_IP}:5432
      PATRONI_SUPERUSER_USERNAME: postgres
      PATRONI_SUPERUSER_PASSWORD: ${PG_SUPERUSER_PASSWORD}
      PATRONI_REPLICATION_USERNAME: replicator
      PATRONI_REPLICATION_PASSWORD: ${PG_REPLICATION_PASSWORD}
      PATRONI_REWIND_USERNAME: rewind_user
      PATRONI_REWIND_PASSWORD: ${PG_REWIND_PASSWORD}
    ports:
      - "5432:5432"
      - "8008:8008"
    volumes:
      - pg-data:/var/lib/postgresql/data
```

```
  haproxy:
    image: haproxy:3.0
    container_name: haproxy
    restart: unless-stopped
    environment:
      N1_IP: ${N1_IP}
      N2_IP: ${N2_IP}
      N3_IP: ${N3_IP}
    ports:
      - "5000:5000"
      - "5001:5001"
      - "7000:7000"
    volumes:
      - ./haproxy/haproxy.cfg:/usr/local/etc/haproxy/haproxy.cfg:ro

volumes:
  etcd-data:
  pg-data:
```

Notas: - `restart: unless-stopped` y el arranque automático de Docker hacen que, al encender una máquina apagada, **el nodo vuelva solo** al clúster. - `--initial-cluster-state=new` solo se aplica la primera vez. Después etcd usa lo que guardó en su volumen. - `init: true` añade un proceso *init* que recoge procesos zombis de PostgreSQL.

8.6 `patroni/Dockerfile`

```
FROM postgres:16-bookworm

RUN apt-get update \
 && apt-get install -y --no-install-recommends python3 python3-venv curl \
 && python3 -m venv /opt/patroni \
 && /opt/patroni/bin/pip install --no-cache-dir "patroni[etcd3,psycopg3]==4.0.4" \
 && apt-get clean \
 && rm -rf /var/lib/apt/lists/*

ENV PATH="/opt/patroni/bin:${PATH}"

COPY patroni.yml /etc/patroni/patroni.yml

USER postgres
EXPOSE 5432 8008
ENTRYPOINT ["patroni", "/etc/patroni/patroni.yml"]
```

> `postgres:16-bookworm` es multi-arquitectura: en la Mac se construye para arm64 y en Pop!_OS para amd64, con la misma versión de PostgreSQL y de glibc.

8.7 `patroni/patroni.yml`

Lo que cambia por nodo (nombre, IP y contraseñas) llega por variables de entorno `PATRONI_*` desde el compose. Este archivo es idéntico en los 3.

```
scope: hadb
namespace: /service/

restapi:
  listen: 0.0.0.0:8008

bootstrap:
  dcs:
    ttl: 20
    loop_wait: 5
    retry_timeout: 5
    maximum_lag_on_failover: 1048576        # 1 MB: una réplica más atrasada no puede ser promovida
    synchronous_mode: true                   # cada COMMIT confirmado está en ≥ 2 nodos
    synchronous_mode_strict: false
```

```
    synchronous_node_count: 1
    postgresql:
      use_pg_rewind: true
      use_slots: true
      parameters:
        wal_level: replica
        hot_standby: "on"
        wal_log_hints: "on"
        max_connections: 100
        max_wal_senders: 10
        max_replication_slots: 10
        wal_keep_size: 1GB                  # margen si un nodo pasa mucho tiempo desconectado
        synchronous_commit: "on"
        password_encryption: scram-sha-256
        log_min_duration_statement: 1000
  initdb:
    - encoding: UTF8
    - locale: C.UTF-8
    - data-checksums

postgresql:
  listen: 0.0.0.0:5432
  data_dir: /var/lib/postgresql/data/patroni
  bin_dir: /usr/lib/postgresql/16/bin
  pgpass: /tmp/pgpass0
  parameters:
    unix_socket_directories: /var/run/postgresql
  pg_hba:
    - local   all          all                       trust
    - host    replication  replicator  0.0.0.0/0     scram-sha-256
    - host    all          all         127.0.0.1/32  scram-sha-256
    - host    all          all         0.0.0.0/0     scram-sha-256

tags:
  nofailover: false
  noloadbalance: false
  clonefrom: false
  nosync: false
```

Parámetros clave para explicar en la defensa:

Parámetro	Valor	Efecto
`ttl`	20 s	Si el líder no renueva su llave en 20 s, se considera muerto.
`loop_wait`	5 s	Cada cuánto revisa Patroni el estado.
`synchronous_mode`	true	Hay una réplica síncrona y RPO = 0.
`maximum_lag_on_failover`	1 MB	No se promueve una réplica muy atrasada.
`use_pg_rewind`	true	El primario viejo se reincorpora sin copia completa.
`use_slots`	true	Slots de replicación: el primario retiene el WAL que una réplica todavía no recibe.
`wal_keep_size`	1 GB	WAL extra retenido. Con esta carga cubre muchas horas de desconexión.
`data-checksums`	sí	Detecta corrupción de páginas.

8.8 `haproxy/haproxy.cfg`

```
global
    maxconn 300

defaults
    mode tcp
    log stdout format raw local0
    retries 2
    timeout connect 4s
    timeout client  30m
    timeout server  30m
```

```
    timeout check   3s

listen stats
    mode http
    bind *:7000
    stats enable
    stats uri /
    stats refresh 2s

# Escritura: SOLO el nodo que responde 200 en /primary
listen primario
    bind *:5000
    option httpchk
    http-check send meth GET uri /primary
    http-check expect status 200
    default-server inter 2s fall 2 rise 2 on-marked-down shutdown-sessions
    server n1 "${N1_IP}:5432" maxconn 100 check port 8008
    server n2 "${N2_IP}:5432" maxconn 100 check port 8008
    server n3 "${N3_IP}:5432" maxconn 100 check port 8008

# Lectura: réplicas sanas, en round-robin
listen replicas
    bind *:5001
    balance roundrobin
    option httpchk
    http-check send meth GET uri /replica
    http-check expect status 200
    default-server inter 2s fall 2 rise 2 on-marked-down shutdown-sessions
    server n1 "${N1_IP}:5432" maxconn 100 check port 8008
    server n2 "${N2_IP}:5432" maxconn 100 check port 8008
    server n3 "${N3_IP}:5432" maxconn 100 check port 8008
```

- `on-marked-down shutdown-sessions` cierra de inmediato las conexiones hacia un nodo que deja de ser primario, para que el cliente reconecte al nuevo.
- Las IP vienen de variables de entorno, así que el archivo es igual en los 3 nodos.

8.9 `sql/esquema.sql`

```
-- Se ejecuta UNA vez, conectado a la BD hadb como usuario app, en el primario.

CREATE TABLE cuentas (
    id          integer       PRIMARY KEY,
    titular     text          NOT NULL,
    saldo       numeric(14,2) NOT NULL CHECK (saldo >= 0),
    actualizado timestamptz   NOT NULL DEFAULT now()
);

CREATE TABLE movimientos (
    id         bigserial     PRIMARY KEY,
    generador  text          NOT NULL,          -- p.ej. "n2-20261105101500"
    seq        bigint        NOT NULL,          -- 1, 2, 3, ... sin huecos por generador
    origen     integer       NOT NULL REFERENCES cuentas(id),
    destino    integer       NOT NULL REFERENCES cuentas(id),
    monto      numeric(14,2) NOT NULL CHECK (monto >= 0),
    primario   text          NOT NULL,          -- IP del primario que aceptó la escritura
    creado     timestamptz   NOT NULL DEFAULT now(),
    CONSTRAINT movimientos_generador_seq_uk UNIQUE (generador, seq)
);

CREATE INDEX movimientos_creado_idx ON movimientos (creado);

-- 100 cuentas con Q10,000.00 cada una -> la suma total SIEMPRE debe ser 1,000,000.00
INSERT INTO cuentas (id, titular, saldo)
SELECT g, 'Cuenta ' || g, 10000
FROM generate_series(1, 100) AS g;
```

Invariantes de integridad que se comprueban en todo momento y en todos los nodos: 1. `SUM(saldo) = 1,000,000.00`. Las transferencias mueven dinero pero no lo crean ni lo destruyen. Si una transacción se aplicara a medias o duplicada, la suma cambiaría. 2. Para cada `generador`, `MAX(seq) = COUNT(*)`, es decir, no falta

ninguna secuencia confirmada. 3. Con la carga detenida y los nodos sincronizados, el `md5` de ambas tablas es idéntico en los 3 nodos.

> Los saltos en `movimientos.id` (por ejemplo de 1520 a 1553) después de un failover **son normales**: PostgreSQL reserva valores de secuencia por bloques de 32 en el WAL. No indican pérdida de datos. La integridad se mide por `(generador, seq)`, no por `id`.

8.10 `app/requirements.txt`

```
psycopg[binary]>=3.2
```

8.11 `app/generador.py`

```
#!/usr/bin/env python3
"""Generador de carga para el clúster HA.

Cada operación es una transferencia entre dos cuentas:
1 INSERT en movimientos + 2 UPDATE en cuentas = 3 cambios por transacción.
Las operaciones son idempotentes (UNIQUE generador+seq): si se cae el primario
a mitad del COMMIT se reintenta la misma seq sin riesgo de duplicarla.
"""
import argparse
import csv
import os
import random
import socket
import time
from datetime import datetime

import psycopg

CUENTAS = 100


def ahora():
    return datetime.now().strftime("%H:%M:%S.%f")[:-3]


def primera_linea(e):
    texto = str(e).strip()
    return texto.splitlines()[0] if texto else type(e).__name__


def conectar(dsn):
    intento = 0
    while True:
        try:
            conn = psycopg.connect(dsn, autocommit=True)
            print(f"{ahora()} conectado al primario {conn.info.host}", flush=True)
            return conn
        except psycopg.Error as e:
            intento += 1
            print(f"{ahora()} sin primario (intento {intento}): {primera_linea(e)}", flush=True)
            time.sleep(min(2.0, 0.5 * intento))


def transferir(conn, generador, seq, origen, destino, monto):
    """Aplica la transferencia. Devuelve 'ok' o 'duplicado'."""
    with conn.transaction():
        # Bloquear ambas cuentas en orden de id evita deadlocks entre generadores
        conn.execute(
            "SELECT id FROM cuentas WHERE id = ANY(%s) ORDER BY id FOR UPDATE",
            ([origen, destino],),
        )
        saldo = conn.execute(
            "SELECT saldo FROM cuentas WHERE id = %s", (origen,)
        ).fetchone()[0]
```

```
        monto = min(monto, saldo)
        fila = conn.execute(
            """INSERT INTO movimientos (generador, seq, origen, destino, monto, primario)
               VALUES (%s, %s, %s, %s, %s, %s)
               ON CONFLICT (generador, seq) DO NOTHING
               RETURNING id""",
            (generador, seq, origen, destino, monto, conn.info.host),
        ).fetchone()
        if fila is None:
            return "duplicado"
        conn.execute(
            "UPDATE cuentas SET saldo = saldo - %s, actualizado = now() WHERE id = %s",
            (monto, origen),
        )
        conn.execute(
            "UPDATE cuentas SET saldo = saldo + %s, actualizado = now() WHERE id = %s",
            (monto, destino),
        )
    return "ok"


def main():
    ap = argparse.ArgumentParser(description="Generador de carga HA")
    ap.add_argument("--tps", type=float, default=1.0,
                    help="transacciones por segundo (1.0 = 60 tx/min = 180 cambios/min)")
    ap.add_argument("--nombre", default=socket.gethostname().split(".")[0],
                    help="prefijo del generador (por defecto el hostname)")
    args = ap.parse_args()

    dsn = os.environ["HA_DSN"]
    generador = f"{args.nombre}-{datetime.now():%Y%m%d%H%M%S}"
    os.makedirs("evidencias", exist_ok=True)
    ruta_csv = os.path.join("evidencias", f"generador_{generador}.csv")

    with open(ruta_csv, "w", newline="") as archivo:
        log = csv.writer(archivo)
        log.writerow(["timestamp", "generador", "seq", "resultado",
                      "primario", "latencia_ms", "intentos"])

        print(f"{ahora()} generador {generador} a {args.tps} tx/s -> {ruta_csv}", flush=True)
        conn = conectar(dsn)
        periodo = 1.0 / args.tps
        siguiente = time.monotonic()
        seq = ok = duplicados = 0

        try:
            while True:
                seq += 1
                origen, destino = random.sample(range(1, CUENTAS + 1), 2)
                monto = random.randint(1, 500)
                intentos = 0
                t0 = time.monotonic()
                while True:
                    intentos += 1
                    try:
                        resultado = transferir(conn, generador, seq, origen, destino, monto)
                        break
                    except psycopg.Error as e:
                        print(f"{ahora()} seq={seq} intento={intentos} error: "
                              f"{type(e).__name__}: {primera_linea(e)}", flush=True)
                        try:
                            conn.close()
                        except Exception:
                            pass
                        conn = conectar(dsn)

                latencia = (time.monotonic() - t0) * 1000
                ok += resultado == "ok"
                duplicados += resultado == "duplicado"
                log.writerow([datetime.now().isoformat(timespec="milliseconds"), generador,
                              seq, resultado, conn.info.host, f"{latencia:.1f}", intentos])
                archivo.flush()
                if intentos > 1 or seq % 30 == 0:
                    print(f"{ahora()} seq={seq} {resultado} primario={conn.info.host} "
```

```
                        f"lat={latencia:.0f}ms intentos={intentos} "
                        f"(ok={ok} duplicados={duplicados})", flush=True)

                # Ritmo constante; si hubo una pausa (failover) se recupera el atraso
                siguiente += periodo
                espera = siguiente - time.monotonic()
                if espera > 0:
                    time.sleep(espera)
                elif espera < -120:
                    siguiente = time.monotonic()
        except KeyboardInterrupt:
            print(f"\n{ahora()} detenido. último seq={seq} ok={ok} duplicados={duplicados}")


if __name__ == "__main__":
    main()
```

Comportamiento esperado en un failover: - Las transacciones en curso fallan, el generador imprime `sin primario (intento N)` y reintenta cada 0.5–2 s. - Al aparecer el nuevo primario reintenta **la misma `seq`**. Si la transacción sí se había guardado, el resultado es `duplicado` y no se aplica dos veces. - Después ejecuta las transacciones atrasadas a máxima velocidad, de modo que el conteo por minuto se recupera.

8.12 `app/monitor.py`

```
#!/usr/bin/env python3
"""Monitor en vivo del clúster: rol de cada nodo, lag, ritmo de cambios e invariantes."""
import csv
import json
import os
import time
import urllib.request
from datetime import datetime

import psycopg

NODOS = {"n1": os.environ["N1_IP"], "n2": os.environ["N2_IP"], "n3": os.environ["N3_IP"]}
DSN = os.environ["HA_DSN"]
INTERVALO = 2


def estado_patroni(ip):
    try:
        with urllib.request.urlopen(f"http://{ip}:8008/patroni", timeout=1.5) as r:
            return json.load(r)
    except Exception:
        return None


def metricas(conn):
    return conn.execute(
        """SELECT count(*) FILTER (WHERE creado > now() - interval '60 seconds'),
                  count(*),
                  (SELECT sum(saldo) FROM cuentas)
           FROM movimientos"""
    ).fetchone()


def main():
    os.makedirs("evidencias", exist_ok=True)
    ruta = os.path.join("evidencias", f"monitor_{datetime.now():%Y%m%d_%H%M%S}.csv")
    conn = None
    with open(ruta, "w", newline="") as f:
        log = csv.writer(f)
        log.writerow(["timestamp", "n1", "n2", "n3", "primario",
                      "tx_ultimo_min", "cambios_ultimo_min", "total_movimientos", "suma_saldos"])
        while True:
            estados = {n: estado_patroni(ip) for n, ip in NODOS.items()}
            lsn_primario = None
            for e in estados.values():
                if e and e.get("role") in ("primary", "master"):
```

```
                    lsn_primario = e.get("xlog", {}).get("location")

            filas = []
            for n, e in estados.items():
                if e is None:
                    filas.append((n, "CAÍDO", "-", "-", "-"))
                    continue
                rol = e.get("role", "?")
                if rol == "master":
                    rol = "primary"
                lag = "-"
                if rol == "replica" and lsn_primario is not None:
                    replay = e.get("xlog", {}).get("replayed_location")
                    if replay is not None:
                        lag = f"{max(0, lsn_primario - replay)} B"
                filas.append((n, rol, e.get("state", "?"), e.get("timeline", "-"), lag))

            try:
                if conn is None or conn.closed:
                    conn = psycopg.connect(DSN, autocommit=True, connect_timeout=2)
                tx_min, total, suma = metricas(conn)
                primario = conn.info.host
            except psycopg.Error:
                conn, tx_min, total, suma, primario = None, None, None, None, "SIN PRIMARIO"

            os.system("clear")
            print(f"Clúster hadb  {datetime.now():%Y-%m-%d %H:%M:%S}\n")
            print(f"{'nodo':5} {'rol':9} {'estado':10} {'TL':>3} {'lag':>10}")
            for n, rol, estado, tl, lag in filas:
                print(f"{n:5} {rol:9} {estado:10} {str(tl):>3} {lag:>10}")
            print()
            print(f"Primario (vía libpq) : {primario}")
            if tx_min is not None:
                marca = "OK" if tx_min * 3 >= 90 else "BAJO"
                integridad = "OK" if suma == 1000000 else "¡ALERTA!"
                print(f"Transacciones / min  : {tx_min}")
                print(f"Cambios / min        : {tx_min * 3}  [{marca}, mínimo 90]")
                print(f"Total movimientos    : {total}")
                print(f"Suma de saldos       : {suma:,.2f}  [{integridad}]")

            roles = {n: r for n, r, *_ in filas}
            log.writerow([datetime.now().isoformat(timespec="seconds"),
                          roles["n1"], roles["n2"], roles["n3"], primario,
                          tx_min, None if tx_min is None else tx_min * 3, total, suma])
            f.flush()
            time.sleep(INTERVALO)


if __name__ == "__main__":
    main()
```

> Patroni 4 reporta el rol del primario como `primary`. Las versiones viejas decían `master` y el monitor acepta ambos.

8.13 `app/verificar.py`

```
#!/usr/bin/env python3
"""Verificador de integridad: compara los 3 nodos directamente (puerto 5432)."""
import os
import sys
import time

import psycopg

NODOS = {"n1": os.environ["N1_IP"], "n2": os.environ["N2_IP"], "n3": os.environ["N3_IP"]}
CLAVE = os.environ["APP_PASSWORD"]
SUMA_ESPERADA = 1000000

CONSULTA = """
SELECT pg_is_in_recovery(),
```

```
       (SELECT count(*) FROM cuentas),
       (SELECT sum(saldo) FROM cuentas),
       (SELECT md5(string_agg(id || ':' || saldo, ',' ORDER BY id)) FROM cuentas),
       (SELECT count(*) FROM movimientos),
       (SELECT md5(string_agg(generador || ':' || seq || ':' || origen || ':' || destino
                              || ':' || monto, ',' ORDER BY generador, seq)) FROM movimientos),
       (SELECT coalesce(sum(faltantes), 0) FROM
          (SELECT max(seq) - count(*) AS faltantes FROM movimientos GROUP BY generador) x),
       CASE WHEN pg_is_in_recovery() THEN pg_last_wal_replay_lsn() ELSE pg_current_wal_lsn() END
"""


def dsn(ip):
    return f"host={ip} port=5432 dbname=hadb user=app password={CLAVE} connect_timeout=3"


def leer(ip):
    try:
        with psycopg.connect(dsn(ip)) as conn:
            return conn.execute(CONSULTA).fetchone()
    except psycopg.Error as e:
        return e


def main():
    esperar = "--esperar" in sys.argv
    if esperar:
        # Esperar a que todas las réplicas alcancen el LSN del primario (carga detenida)
        for _ in range(30):
            datos = {n: leer(ip) for n, ip in NODOS.items()}
            lsns = {n: d[7] for n, d in datos.items() if not isinstance(d, Exception)}
            if len(set(str(v) for v in lsns.values())) == 1:
                break
            time.sleep(1)

    datos = {n: leer(ip) for n, ip in NODOS.items()}
    print(f"{'nodo':5} {'rol':8} {'cuentas':>7} {'suma':>14} {'movs':>8} {'faltan':>6}  "
          f"{'md5 cuentas':12} {'md5 movs':12} lsn")
    errores = []
    vivos = {}
    for n, d in datos.items():
        if isinstance(d, Exception):
            print(f"{n:5} NO DISPONIBLE ({str(d).strip().splitlines()[0] if str(d).strip() else
type(d).__name__})")
            continue
        replica, ncuentas, suma, h_cuentas, nmovs, h_movs, faltan, lsn = d
        rol = "réplica" if replica else "PRIMARIO"
        vivos[n] = d
        print(f"{n:5} {rol:8} {ncuentas:>7} {suma:>14,.2f} {nmovs:>8} {faltan:>6}  "
              f"{(h_cuentas or '')[:12]:12} {(h_movs or '')[:12]:12} {lsn}")
        if suma != SUMA_ESPERADA:
            errores.append(f"{n}: suma de saldos {suma} != {SUMA_ESPERADA}")
        if faltan != 0:
            errores.append(f"{n}: faltan {faltan} secuencias")

    primarios = [n for n, d in vivos.items() if not d[0]]
    if len(primarios) != 1:
        errores.append(f"hay {len(primarios)} primarios: {primarios}")
    if len({(d[3], d[5]) for d in vivos.values()}) > 1:
        errores.append("los nodos NO tienen datos idénticos "
                       "(normal si la carga sigue corriendo; detenerla y usar --esperar)")

    print()
    if errores:
        print("RESULTADO: FALLA")
        for e in errores:
            print("  -", e)
        sys.exit(1)
    print(f"RESULTADO: OK. {len(vivos)} nodos con datos idénticos, suma = {SUMA_ESPERADA:,}, "
          f"sin secuencias faltantes, un solo primario.")


if __name__ == "__main__":
    main()
```

8.14 `scripts/probar_puertos.sh`

```
#!/usr/bin/env bash
# Prueba desde ESTE nodo que los puertos del clúster respondan en los 3 nodos.
set -u
cd "$(dirname "$0")/.." || exit 1
set -a; source .env; set +a

for ip in "$N1_IP" "$N2_IP" "$N3_IP"; do
  for p in 2379 2380 5432 8008 5000 7000; do
    if nc -z -w 2 "$ip" "$p" 2>/dev/null; then
      echo "OK     $ip:$p"
    else
      echo "FALLA  $ip:$p"
    fi
  done
done
```

```
chmod +x scripts/probar_puertos.sh
```

Criterio de salida de la fase 3: el repo está clonado en las 3 máquinas, cada una tiene su `.env` correcto y `git status` no muestra `.env` como archivo a subir.

---

9. Fase 4: Clúster etcd

9.1 Construir la imagen de Patroni (las 3, se puede hacer ya)

```
cd ~/hadb
docker compose build patroni      # 2–5 min la primera vez
docker compose pull etcd haproxy
```

9.2 Arrancar etcd en los 3 nodos

Ejecutarlo en las 3 máquinas con pocos minutos de diferencia. etcd espera a sus pares para formar quórum:

```
cd ~/hadb
docker compose up -d etcd
docker compose logs -f etcd       # Ctrl+C para salir
```

Mensajes buenos: `became leader at term` en uno y `elected leader` en los otros.

9.3 Verificación

```
docker exec etcd etcdctl --endpoints=http://$N1_IP:2379,http://$N2_IP:2379,http://$N3_IP:2379 endpoint
health
docker exec etcd etcdctl --endpoints=http://$N1_IP:2379,http://$N2_IP:2379,http://$N3_IP:2379 endpoint
status -w table
docker exec etcd etcdctl member list -w table
```

> Para usar `$N1_IP` en la terminal, cargar antes el `.env`: `set -a; source .env; set +a`.

Esperado: 3 endpoints `is healthy` y uno de ellos con `IS LEADER = true`.

Criterio de salida: los 3 miembros sanos y un líder.

---

10. Fase 5: PostgreSQL + Patroni

10.1 Arranque ordenado (solo la primera vez)

1. **n2 primero** (Linux, para que el primario inicial sea x86_64; da igual en la práctica): `bash docker compose up -d patroni docker compose logs -f patroni` Esperar a ver `initialized a new cluster` y luego `no action. I am (n2), the leader with the lock`.
2. **n3 y luego n1**: `bash docker compose up -d patroni docker compose logs -f patroni` Esperar a ver `bootstrapped from leader 'n2'` (copia con `pg_basebackup`) y después `no action. I am (n3), a secondary, and following a leader (n2)`.

10.2 Verificación del clúster

```
docker exec -it patroni patronictl -c /etc/patroni/patroni.yml list
```

Esperado (los nombres de host serán las IP):

```
+ Cluster: hadb (7432...) ----+--------------+-----------+----+-----------+
| Member | Host          | Role         | State     | TL | Lag in MB |
+--------+---------------+--------------+-----------+----+-----------+
| n1     | 100.64.0.1    | Replica      | streaming |  1 |         0 |
| n2     | 100.64.0.2    | Leader       | running   |  1 |           |
| n3     | 100.64.0.3    | Sync Standby | streaming |  1 |         0 |
+--------+---------------+--------------+-----------+----+-----------+
```

Hay que ver exactamente **1 Leader**, **1 Sync Standby** y **1 Replica**, todos con lag 0.

10.3 Verificar la API REST de Patroni (desde cualquier nodo)

```
for ip in $N1_IP $N2_IP $N3_IP; do
  echo -n "$ip /primary -> "; curl -s -o /dev/null -w "%{http_code}\n" http://$ip:8008/primary
done
curl -s http://$N2_IP:8008/cluster | jq
```

Solo un nodo debe responder `200` en `/primary`. Los demás responden `503`.

10.4 Verificar la replicación desde PostgreSQL

En el primario:

```
docker exec -it patroni psql -U postgres -c \
 "SELECT application_name, client_addr, state, sync_state, replay_lag FROM pg_stat_replication;"
```

Esperado: 2 filas, una `sync` y otra `async`, ambas en estado `streaming`.

10.5 Prueba de compatibilidad arm64 ↔ x86_64 (puerta de control)

Esto es lo que más riesgo técnico tiene, así que se valida ya. En n1 (la Mac) los logs deben mostrar que la réplica arrancó y está en `streaming`. Para confirmarlo:

```
docker exec -it patroni psql -U postgres -c "SELECT pg_is_in_recovery(), pg_last_wal_replay_lsn();"
```

Debe responder `t` y un LSN que avanza. Luego se cambia el primario hasta que un nodo de cada arquitectura lo haya sido:

```
./scripts/pctl.sh switchover --force    # el Sync Standby pasa a primario
./scripts/pctl.sh list                  # esperar running/streaming con lag 0; repetir si hace falta
```

> **Importante:** con replicación síncrona, Patroni **solo** acepta como nuevo primario a la réplica que en ese momento es `Sync Standby` (si no, responde `412 candidate name does not match with sync_standby`). Es una protección: así nunca se promueve un nodo al que le falten datos confirmados. Por eso el cambio se hace **sin `--candidate`** (Patroni elige a la síncrona) y se repite hasta que el nodo deseado sea `Leader`.

Si cada cambio termina con los 3 nodos en `streaming` / `running`, la combinación funciona.

> ✓ **Resultado real (2026-10-23):** n3 (Mac ARM, OrbStack) arrancó como primario y n2 (Pop!_OS x86) replicó desde él. Después n3 → n1 → n2, y con n2 (x86) como primario las dos Mac replicaron con los datos intactos. La compatibilidad arm64 ↔ x86_64 quedó validada en ambas direcciones.

10.6 Plan B si la Mac no puede replicar con los nodos x86_64

Síntomas: en n1 aparecen errores como `database files are incompatible with server` o `WAL file is from different database system`, o la réplica no arranca.

Solución: correr el contenedor de Patroni de la Mac en amd64 emulado con Rosetta. Así los 3 nodos tienen exactamente el mismo binario. 1. En la Mac, Docker Desktop: verificar que "Use Rosetta for x86_64/amd64 emulation" esté activo. 2. En `docker-compose.yml`, agregar en el servicio `patroni` (se puede hacer con un `docker-compose.override.yml` **solo en la Mac**, sin tocar el compose común): `yaml # docker-compose.override.yml (solo en n1; agregarlo a .gitignore) services: patroni: platform: linux/amd64` 3. Reconstruir y reinicializar ese nodo: `bash docker compose down docker volume rm hadb_pg-data docker compose build --no-cache patroni docker compose up -d` El rendimiento con emulación es menor, pero sobra para 180 transacciones por minuto.

Criterio de salida de la fase 5: `patronictl list` muestra Leader, Sync Standby y Replica, y el *switchover* de ida y vuelta con la Mac funcionó.

---

11. Fase 6: HAProxy

En los 3 nodos:

```
docker compose up -d haproxy
docker compose ps                 # etcd, patroni, haproxy en estado "running"
./scripts/probar_puertos.sh       # todo "OK"
```

Ver el panel de estadísticas en el navegador: `http://<IP_de_cualquier_nodo>:7000`. En el bloque `primario` debe haber una sola línea en verde, la del líder. En `replicas` deben aparecer dos en verde.

Prueba con `psql` por HAProxy desde cualquier nodo:

```
docker exec -it patroni psql "host=$NODE_IP port=5000 user=postgres password=$PG_SUPERUSER_PASSWORD
dbname=postgres" \
  -c "SELECT inet_server_addr(), pg_is_in_recovery();"
```

`pg_is_in_recovery` debe ser `f`, porque el puerto 5000 llega al primario.

Criterio de salida de la fase 6: el panel muestra 1 primario y 2 réplicas en verde desde los 3 nodos.

---

12. Fase 7: Base de datos, esquema y usuario de la aplicación

Solo **una vez**, en el nodo que sea el primario:

```
cd ~/hadb
set -a; source .env; set +a

docker exec -i patroni psql -U postgres <<SQL
CREATE ROLE app LOGIN PASSWORD '${APP_PASSWORD}';
CREATE DATABASE hadb OWNER app;
SQL

docker exec -i patroni psql "host=127.0.0.1 dbname=hadb user=app password=${APP_PASSWORD}" < sql/
esquema.sql
```

Verificar en **cada** nodo que el esquema llegó por replicación:

```
docker exec -it patroni psql -U postgres -d hadb -c "SELECT count(*), sum(saldo) FROM cuentas;"
```

Esperado en los 3 nodos: `100 | 1000000.00`.

---

13. Fase 8: Generador de carga, monitor y verificador

13.1 Entorno Python (las 3 máquinas)

```
cd ~/hadb
python3 -m venv .venv
source .venv/bin/activate
pip install -r app/requirements.txt
```

13.2 Variables de entorno de la aplicación

En cada terminal donde se corra un script:

```
cd ~/hadb
source .venv/bin/activate
set -a; source .env; set +a
export HA_DSN="host=$N1_IP,$N2_IP,$N3_IP port=5432 dbname=hadb user=app password=$APP_PASSWORD \
target_session_attrs=read-write connect_timeout=3 \
keepalives=1 keepalives_idle=5 keepalives_interval=2 keepalives_count=3 tcp_user_timeout=10000"
```

Para no repetirlo, guardar las líneas en `scripts/env_app.sh` (sin contraseñas en el archivo, porque las toma del `.env`) y hacer `source scripts/env_app.sh`.

Parámetro de la cadena	Para qué
`host=a,b,c`	libpq prueba los 3 nodos en orden
`target_session_attrs=read-write`	se queda solo con el que acepta escrituras (el primario)
`connect_timeout=3`	no espera más de 3 s por un nodo apagado
`keepalives*`, `tcp_user_timeout`	detectan en unos 10 s una conexión a un nodo que murió sin cerrar el socket

13.3 Ejecución (una terminal por script)

Terminal	Nodo	Comando
A	n1, n2, n3 (cada uno)	`python app/generador.py --nombre n1` (y `n2`, `n3` según el nodo)
B	el que se proyecte en la demo	`./scripts/panel.sh` (panel web, `http://localhost:8080`) o `python app/monitor.py` (terminal)
C	cualquiera	`python app/verificar.py` (cuando se quiera)
D	cualquiera	`watch -n 2 docker exec patroni patronictl -c /etc/patroni/patroni.yml list`

> En macOS `watch` se instala con `brew install watch`.

13.4 Prueba de humo

1. Arrancar los 3 generadores.
2. En el monitor, tras 60 s: unas **180 tx/min** y **540 cambios/min**, `Suma = 1,000,000.00 [OK]`.
3. Correr `python app/verificar.py`. La carga está activa, así que los md5 pueden diferir por milisegundos de lag, pero la suma y "faltan 0" deben estar bien en los 3.
4. Detener los 3 generadores con `Ctrl+C` y correr `python app/verificar.py --esperar`. El resultado debe ser `RESULTADO: OK. 3 nodos con datos idénticos...`.

Criterio de salida de la fase 8: la prueba de humo pasa y el verificador da OK.

---

14. Fase 9: Pruebas de falla

14.1 Procedimiento común para cada prueba

1. Clúster sano: `patronictl list` con 3 nodos y lag 0, y los 3 generadores corriendo al menos 2 minutos.
2. Anotar la hora exacta (`date +%T`) y quién es el primario.
3. Provocar la falla.
4. Observar el monitor y `patronictl list`. Anotar cuándo vuelve a haber primario (RTO) y que la suma siga en 1,000,000.
5. Esperar 2 minutos con el nodo caído. El ritmo debe seguir por encima de 90 cambios/min.
6. Restaurar el nodo. Anotar cuándo vuelve a `streaming` con lag 0.
7. Detener los generadores y correr `verificar.py --esperar`.
8. Guardar evidencia: capturas, `docker compose logs patroni > evidencias/pX_nY.log` y los CSV.

14.2 Matriz de pruebas

ID	Escenario	Cómo provocarlo	Resultado esperado	RTO esperado
P1	Réplica apagada limpiamente	En una réplica: `sudo shutdown -h now` (Linux) o *Apagar* (Mac)	El primario sigue. Si era la síncrona, Patroni promueve la otra réplica a síncrona. Los generadores no se interrumpen.	0 s (puede haber un *stall* de 1–5 s si era la síncrona)
P2	Réplica apagada de golpe	Mantener presionado el botón de encendido, o desconectar la corriente en un equipo sin batería	Igual que P1.	0–10 s
P3	**Primario apagado de golpe**	Botón de encendido del primario	Failover automático: la réplica síncrona pasa a Leader y la otra se convierte en Sync Standby. Los generadores sobrevivientes reintentan y continúan.	20–30 s (laboratorio: 22 s)
P4	**Primario sin red (partición)**	Desconectar WiFi/cable del primario, o `sudo tailscale down` en él	El aislado pierde el quórum de etcd y se **degrada solo** a réplica de solo lectura. Los otros dos eligen un primario nuevo. No hay *split-brain*. Al reconectar, el viejo hace `pg_rewind` y sigue al nuevo.	20–30 s (laboratorio: 27 s)
P5	Réplica sin red	Desconectar la red de una réplica	Como P1. Al reconectar recupera el WAL atrasado.	0–5 s
P6	Reinicio completo de un nodo	`sudo reboot`	Al arrancar, Docker levanta los contenedores y el nodo vuelve como réplica **sin intervención**.	Depende del arranque del SO
P7	Desconexión prolongada	Nodo apagado de 30 a 60 min con carga	Al volver, sincroniza con el WAL retenido. Si no le alcanzara, Patroni hace `reinit` (copia completa).	n/a
P8	Switchover planificado	`patronictl switchover`	Cambio de primario controlado, unos 5 s sin escrituras.	2–6 s
P9	Docker detenido en el primario	`docker compose stop patroni`	Igual que P3, pero Patroni libera la llave de inmediato, así que es más rápido.	2–8 s
P10	*(Fuera de alcance)* 2 nodos caídos	Apagar 2 nodos	Sin quórum de etcd: el nodo restante queda **solo lectura**. Es lo correcto: protege la integridad.	n/a

> P10 se documenta para explicar el límite del diseño. Con N nodos se toleran $\lfloor (N-1)/2 \rfloor$ fallas: con 3 se tolera 1.

14.3 Qué se observa en cada prueba (para explicarlo)

- **Generador** del nodo que se apaga: muere con su equipo. Es esperado y no deja huecos, porque su `seq` simplemente termina.
- **Generadores sobrevivientes:** imprimen `sin primario (intento N)` durante el failover. Luego `seq=... intentos=K` y a veces `duplicado`, lo que demuestra la idempotencia.
- **Monitor:** el nodo caído aparece como `CAÍDO`, el TL (timeline) sube en 1 tras el failover y los cambios por minuto bajan unos segundos y se recuperan.
- **`patronictl history`** muestra cada cambio de timeline y su motivo.

14.4 Tabla para registrar resultados (copiar al informe)

Prueba	Fecha/ hora	Primario antes	Nodo afectado	Inicio falla	Nuevo primario a las	RTO (s)	Suma OK	Faltan 0	Resync OK a las	Verificador
P1										
P2										
P3										
P4										
…										

15. Fase 10: Guion de la demostración en vivo

15.1 Día anterior

- ☐ Las 3 máquinas cargadas, con cargador y sin reposo (§5).
- ☐ Tailscale con sesión iniciada en las 3 y expiración de llave desactivada.
- ☐ Router o hotspot de respaldo cargado.
- ☐ Ensayo completo de P3 y P4 por lo menos una vez **en la red donde será la demo** (si es posible).
- ☐ `git pull` en las 3 (misma versión de todo).
- ☐ Limpiar datos de ensayos (opcional): con todo sano, desde el primario, `TRUNCATE movimientos; UPDATE cuentas SET saldo = 10000;` (`TRUNCATE` también se replica).

15.2 30 minutos antes

1. Conectar las 3 a la red y verificar `tailscale ping` entre todas.
2. Encender todo: `docker compose up -d` en las 3 (si ya estaba, no hace nada).
3. `patronictl list`: Leader, Sync Standby y Replica con lag 0.
4. Abrir en la máquina que se proyecta: - Navegador: **panel web** (`./scripts/panel.sh` → `http://localhost:8080`) en pantalla grande. Muestra roles, cambios por minuto, suma de saldos, eventos y la duración de cada failover. - Terminal 1 (respaldo): `monitor.py`. - Terminal 2: `watch -n 2 ... patronictl list`. - Navegador: panel HAProxy `http://<IP>:7000`.
5. Arrancar los 3 generadores.

> **Proyectar desde una máquina que NO sea la que se va a apagar.** Como no se sabe cuál apagarán, conviene tener el monitor abierto en **dos** máquinas, o proyectar desde un cuarto equipo (otra laptop o un celular con SSH) que solo observe.

15.3 Guion (unos 15 min)

Min	Acción	Qué decir
0–2	Mostrar el diagrama de arquitectura	"3 nodos en 3 máquinas físicas, dos sistemas operativos y dos arquitecturas de CPU."
2–4	Monitor + `patronictl list`	"Un primario, una réplica síncrona y una asíncrona. Hay 540 cambios por minuto, el requisito es 90."
4–5	`verificar.py`	"La suma de saldos es invariante, no faltan secuencias y los 3 nodos tienen los mismos datos."
5–6	Escribir desde **cada** máquina	Mostrar `psql -p 5000` en n1, n2 y n3: las 3 escriben al primario.
6–9	**El catedrático apaga una máquina**	Narrar en vivo: "La llave de líder expira en 20 s, etcd con quórum 2/3 elige un primario nuevo, el timeline sube y los generadores reintentan."
9–10	Monitor	"La carga sigue sobre 90 por minuto y la suma sigue en un millón: no se perdió ni se duplicó nada."
10–12	**Encender la máquina**	"Docker levanta los contenedores solos. Patroni detecta un timeline viejo, aplica `pg_rewind` y vuelve como réplica."
12–14	Detener generadores, `verificar.py --esperar`	"Los 3 nodos tienen hashes idénticos: re-sincronización completa."
14–15	`patronictl history`	Mostrar el historial de failovers.

15.4 Si algo sale mal en la demo

Síntoma	Acción inmediata
El nodo que regresa no se une (queda en `stopped` o `start failed`)	`docker exec patroni patronictl -c /etc/patroni/patroni.yml reinit hadb <nodo> --force`
Ningún nodo es Leader	Revisar etcd: `etcdctl endpoint health`. Si hay 2 de 3 sanos, el líder aparece solo en menos de 30 s.
La Mac se durmió	Despertarla. Vuelve sola como réplica. Explicarlo como una falla adicional real.
Generador detenido con error de esquema	Revisar que `HA_DSN` y `APP_PASSWORD` estén cargados en esa terminal.

16. Fase 11: Informe y evidencias

16.1 Estructura sugerida del informe

1. **Portada**
2. **Introducción y objetivos**: los requerimientos R1–R5.
3. **Marco teórico breve**: replicación física en streaming, síncrona vs asíncrona, consenso Raft y quórum, *split-brain*, RPO y RTO, teorema CAP. Este diseño elige consistencia y tolerancia a particiones: el lado en minoría deja de aceptar escrituras.
4. **Arquitectura**: el diagrama de §2.1, la tabla de componentes y las decisiones de §3.
5. **Entorno**: tabla de las 3 máquinas (modelo, SO, CPU, RAM, IP) y la red usada.
6. **Implementación**: un resumen por fase con los archivos de configuración clave.
7. **Diseño de la prueba de carga e integridad**: modelo de cuentas y transferencias, idempotencia e invariantes.
8. **Resultados de pruebas**: la tabla de §14.4 llena, con capturas y gráficas.
9. **Análisis**: RTO medido, RPO = 0 demostrado, comportamiento ante partición.
10. **Limitaciones y mejoras**: más de 1 falla simultánea, *watchdog* de hardware, TLS, respaldos con pgBackRest, monitoreo con Prometheus y Grafana.
11. **Conclusiones**
12. **Anexos**: archivos completos y logs.

16.2 Evidencias que hay que capturar

Evidencia	Cómo
Clúster sano	Captura de `patronictl list` y del panel de HAProxy
Ritmo ≥ 90 cambios/min	Captura del monitor y CSV `evidencias/monitor_*.csv`
Failover	Capturas de `patronictl list` antes, durante y después, y de `patronictl history`
Logs	`docker compose logs --since 30m patroni > evidencias/patroni_nX.log` en cada nodo
Integridad	Salida de `verificar.py --esperar` tras cada prueba
Idempotencia	Líneas `duplicado` / `intentos>1` del CSV del generador
Video	Grabación de pantalla de la prueba P3 o P4 (QuickTime en Mac, `Ctrl+Alt+Shift+R` en Pop!_OS/GNOME)

16.3 Gráfica de cambios por minuto (opcional, para el informe)

Con el CSV del monitor (`cambios_ultimo_min` en el tiempo) se hace en Excel o Google Sheets una gráfica de línea con una línea horizontal en 90. Ahí se ve la caída breve durante el failover y la recuperación. También se puede calcular desde la BD:

```
SELECT date_trunc('minute', creado) AS minuto,
       count(*)      AS transacciones,
       count(*) * 3  AS cambios
FROM movimientos
GROUP BY 1
ORDER BY 1;
```

17. Solución de problemas

Problema	Causa probable	Solución
etcd: `request cluster ID mismatch`	Volumen de etcd de un intento anterior con otra configuración	En **las 3**: `docker compose down`, `docker volume rm hadb_etcd-data` y volver a §9.2. Si había datos de PG, seguir §6.4 desde el paso 5.
etcd no forma quórum	Puertos 2379/2380 bloqueados o IP equivocadas en `.env`	`./scripts/probar_puertos.sh`, revisar `NODE_IP` y los firewalls
Patroni: `waiting for leader to bootstrap` indefinidamente	Ningún nodo pudo inicializar o etcd no está sano	`docker compose logs patroni` en el primer nodo. Comprobar `etcdctl endpoint health`.
Réplica: `pg_basebackup: error: could not connect`	El primario no es alcanzable en 5432 desde ese nodo, o la contraseña de `replicator` difiere	Prueba de §7.3 y comparar `.env`
Réplica: `password authentication failed for user "replicator"`	Contraseñas distintas entre `.env`	Igualar los `.env`. Luego `patronictl reinit hadb <nodo>`.
El nodo que regresa queda en `start failed`	`pg_rewind` falló o faltan WAL	`patronictl reinit hadb <nodo> --force` (copia completa desde el primario)
`patronictl` no encuentra etcd	No toma las variables de entorno	`patronictl -c /etc/patroni/patroni.yml --dcs-url etcd3://$N1_IP:2379 list`
La Mac no recibe conexiones por la IP de Tailscale	Firewall de macOS o Docker Desktop no publica en esa interfaz	Permitir Docker en el firewall. Probar `nc -vz <IP_mac> 5432` desde otro nodo. Como alternativa, instalar OrbStack en lugar de Docker Desktop (mismo `docker compose`).
Mac: 5000 y 7000 ocupados por `ControlCenter`	Receptor AirPlay de macOS (Monterey o posterior)	*Ajustes del Sistema → General → AirDrop y Handoff → Receptor AirPlay* apagado
HAProxy marca todos en rojo	No llega al 8008 o la IP en el `.env` está mal	`curl http://<IP>:8008/primary` desde ese nodo
Monitor: cambios/min < 90	Generadores detenidos o lentos por latencia alta (DERP)	Subir `--tps 2` en los generadores
Escrituras se quedan colgadas	Replicación síncrona sin réplica disponible durante un instante	Se resuelve sola cuando Patroni reasigna la síncrona (≤ 10 s)
Docker en Pop!_OS no arranca tras reiniciar	Servicio deshabilitado	`sudo systemctl enable --now docker`
Docker en Mac no arranca tras reiniciar	Falta el inicio automático	*Settings → General → Start Docker Desktop when you sign in*

18. Comandos de referencia rápida

```
# --- Siempre primero, en la carpeta del proyecto ---
cd ~/hadb && set -a && source .env && set +a
alias pctl='docker exec -it patroni patronictl -c /etc/patroni/patroni.yml'

# Estado
pctl list                       # miembros, rol, lag, timeline
pctl history                    # historial de failovers
pctl topology                   # árbol de replicación
docker compose ps
docker compose logs -f --tail 100 patroni

# Operaciones
pctl switchover --force                              # cambio planificado (al Sync Standby)
pctl failover --candidate n3 --force                  # forzar failover
pctl reinit hadb n1 --force                           # reconstruir un nodo desde el primario
pctl restart hadb n1                                  # reiniciar PostgreSQL en un nodo
pctl show-config                                      # configuración dinámica (DCS)
pctl edit-config                                      # cambiarla (se aplica a los 3)

# etcd
docker exec etcd etcdctl --endpoints=http://$N1_IP:2379,http://$N2_IP:2379,http://$N3_IP:2379 endpoint
status -w table

# PostgreSQL
docker exec -it patroni psql -U postgres -d hadb
#   SELECT * FROM pg_stat_replication;          (en el primario)
#   SELECT pg_is_in_recovery();                 (t = réplica)
#   SELECT * FROM pg_stat_wal_receiver;         (en una réplica)

# API REST de Patroni
curl -s http://$N1_IP:8008/patroni | jq
curl -s http://$N1_IP:8008/cluster | jq

# Ciclo de vida
docker compose up -d            # levantar todo en este nodo
docker compose stop             # detener sin borrar
docker compose down             # detener y quitar contenedores (los datos quedan en volúmenes)
docker compose down -v          # ¡BORRA los datos de ESTE nodo! (solo para reiniciar desde cero)
```

Reinicio total desde cero (solo si se quiere empezar de nuevo)

En las 3: `docker compose down -v`. Luego repetir desde §9.2.

19. Preguntas probables del catedrático

1. **¿Por qué 3 máquinas y no 2?** Para tener quórum. Con 2 no se puede distinguir "el otro murió" de "se cortó la red", y eso termina en *split-brain*. Con 3 decide la mayoría.
2. **¿Qué pasa si se corta la red pero el primario sigue encendido?** Pierde contacto con etcd y no puede renovar su llave de líder, así que Patroni lo degrada a solo lectura antes de que expire el `ttl`. Los otros dos, que sí tienen quórum, eligen un primario nuevo. Nunca hay dos primarios aceptando escrituras. Además, con replicación síncrona el aislado tampoco puede confirmar `COMMIT`, porque no tiene réplica síncrona alcanzable.
3. **¿Se pierden datos en el failover?** No se pierde nada que se haya confirmado al cliente (RPO = 0), porque cada `COMMIT` confirmado está en el primario y en la réplica síncrona. Patroni promueve a la réplica síncrona.
4. **¿Y si la conexión se cae justo en el `COMMIT`?** El cliente no sabe si se guardó. Reintenta con la misma llave `(generador, seq)` y `ON CONFLICT DO NOTHING` evita el duplicado. Se ve como `duplicado` en el log.
5. **¿Cómo se re-sincroniza el nodo que vuelve?** Si era réplica, pide el WAL que le falta desde su última posición. Si era primario, `pg_rewind` descarta lo que escribió y nunca se replicó, y luego sigue al primario nuevo. Si falta WAL, se hace `reinit` con `pg_basebackup`.
6. **¿Por qué saltan los `id` después del failover?** Las secuencias se registran en el WAL por bloques de 32. No es pérdida de datos.
7. **¿Cuánto tarda el failover?** Unos 20–30 s si el primario se apaga de golpe: el `ttl` de 20 s se cuenta desde la última renovación de la llave, más hasta un `loop_wait` (5 s) para detectarlo. En el laboratorio se midieron 22 s (apagado) y 27 s (partición). Si se detiene con limpieza, 2–8 s.
8. **¿Por qué Docker?** Da el mismo binario y la misma glibc en macOS y Linux, lo que evita incompatibilidades de colación en la replicación física, y la configuración es idéntica y reproducible en los 3 nodos.
9. **¿Cómo comprueban la integridad?** Con invariantes de negocio (suma de saldos constante), continuidad de secuencias por generador y hashes md5 idénticos entre los 3 nodos.
10. **¿Qué mejorarían para producción?** TLS en PostgreSQL, etcd y la API de Patroni, respaldos y PITR con pgBackRest, *watchdog*, 5 nodos para tolerar 2 fallas, monitoreo con Prometheus y alertas.

---

20. Cronograma sugerido y checklist final

20.1 Cronograma

Día	Trabajo	Duración aprox.
1	Fases 0–2: SO, red y Docker en las 3 máquinas	2–3 h
1	Fase 3: repositorio y archivos	1 h
2	Fases 4–6: etcd, Patroni y HAProxy, más la prueba arm64 ↔ x86_64	2–3 h
2	Fases 7–8: esquema, generador, monitor y verificador	1–2 h
3	Fase 9: matriz de pruebas P1–P9 con evidencias	3 h
3	Ensayo de la demo en red distinta (si aplica)	1 h
4	Fase 11: informe	3–4 h

20.2 Checklist final

Infraestructura - ☐ Las 3 máquinas sin reposo, con hora sincronizada y Docker con arranque automático - ☐ Red Tailscale (o LAN fija) probada; plan B de red listo - ☐ Los puertos 2379, 2380, 5432, 5000, 5001, 7000 y 8008 responden entre todos los nodos - ☐ etcd: 3 miembros sanos - ☐ Patroni: Leader, Sync Standby y Replica con lag 0 - ☐ La Mac probada como primario y como réplica (switchover de ida y vuelta) - ☐ HAProxy: 1 primario y 2 réplicas en verde en los 3 paneles

Aplicación - ☐ El esquema existe y la suma es 1,000,000 en los 3 nodos - ☐ 3 generadores corriendo: ≥ 180 tx/min y ≥ 540 cambios/min - ☐ El monitor muestra roles, lag, ritmo y suma - ☐ `verificar.py --esperar` da OK

Pruebas - ☐ P1–P9 ejecutadas, registradas en la tabla y con evidencias - ☐ P3 y P4 ensayadas al menos dos veces cada una - ☐ Al menos una prueba con cada máquina (incluida la Mac) como la que se apaga

Entrega - ☐ Informe completo con capturas, tablas y gráficas - ☐ Repositorio actualizado en GitHub en las 3 máquinas (sin `.env`) - ☐ Demo ensayada con el guion de §15

---

21. Laboratorio local: los 3 nodos en una sola máquina

`laboratorio/` levanta el clúster completo en **un solo equipo** con Docker: 3 nodos (cada uno con su etcd, Patroni/PostgreSQL y HAProxy), más un contenedor `app` para el generador, el monitor y el verificador. Usa **la misma imagen y los mismos archivos** (`patroni/`, `haproxy/haproxy.cfg`, `sql/esquema.sql`, `app/`) que el clúster real. Si algo funciona en el laboratorio, la configuración es correcta, y lo único que cambia en las 3 PC es la red.

Sirve para: - Ensayar la demo y las pruebas de falla sin tener las 3 máquinas juntas. - Validar cambios de configuración antes de llevarlos a las 3 PC.

Comando	Qué hace
`./laboratorio/lab.sh iniciar`	Construye la imagen, levanta los 3 nodos, espera primario y réplicas, y crea la BD
`./laboratorio/lab.sh carga`	3 generadores a 1 tx/s (logs en `evidencias/lab_generador_n*.log`)
`./laboratorio/lab.sh monitor`	El mismo `monitor.py` del clúster real
`./laboratorio/lab.sh panel`	El panel web en `http://localhost:18080`
`./laboratorio/lab.sh apagar nX`	Apagado brusco (equivale a desconectar la corriente)
`./laboratorio/lab.sh detener nX`	Apagado limpio
`./laboratorio/lab.sh aislar nX`	Quita la red al nodo sin apagarlo (partición)
`./laboratorio/lab.sh encender nX`	Lo vuelve a encender o reconectar
`./laboratorio/lab.sh estado` / `primario`	`patronictl list` / qué nodo es el primario
`./laboratorio/lab.sh parar-carga` / `verificar`	Detiene generadores / compara los 3 nodos
`./laboratorio/lab.sh destruir`	Borra todo el laboratorio y sus datos

Paneles de HAProxy del laboratorio: `http://localhost:17000` (n1), `:27000` (n2) y `:37000` (n3).

21.1 Resultados medidos en el laboratorio (Mac Apple Silicon, Docker Desktop)

Prueba	Resultado
Arranque	Leader + Sync Standby + Replica en streaming. La réplica síncrona se asigna en el primer ciclo de Patroni (unos 5 s después del arranque).
Carga	180 tx/min (540 cambios/min) con 3 generadores. Suma = 1,000,000.00
P3: primario apagado de golpe	Nuevo primario en **~22 s**. Generadores: 1 error de conexión, 5 intentos sin primario y luego `seq` 67 confirmada en el nuevo primario (`intentos=2`). El minuto siguiente volvió a 180 tx/min por la recuperación del atraso.
Reingreso del primario viejo	`pg_rewind: servers diverged at WAL location ... Done!` y luego réplica en streaming en el timeline 2
P4: primario aislado de la red	Nuevo primario en **~27 s**. El aislado se degradó solo (PostgreSQL reiniciando como réplica): no hubo dos primarios. Al reconectarlo volvió como réplica.
Verificador después de cada prueba	`RESULTADO: OK. 3 nodos con datos idénticos, suma = 1,000,000, sin secuencias faltantes, un solo primario.`

> En los logs de Patroni aparecen avisos `ConnectionResetError` cada pocos segundos. Son los chequeos de salud de HAProxy, que cierran la conexión al terminar. Son inofensivos.